Algorithm for file updates in Python

Project description

In this project, I developed a Python algorithm to automate the process of updating an access control list stored in a text file. The script opens `"allow_list.txt"`, reads its contents, and converts the raw text into a list of IP addresses to enable item-level manipulation. It then iterates through a separate remove list to identify and delete matching IP addresses using conditional logic and the `.remove()` method before writing the updated list back to the file.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement

with open(import_file, "r") as file:
```

To open the allow list file, the string `"allow_list.txt"` is first assigned to the variable `import_file` to hold the target file path. Next, a `with` statement is used alongside the built-in `open()` function, passing `import_file` and the read mode parameter `"r"` to safely open the file. Finally, the keyword `as` assigns the opened file object to the variable `file`, ensuring Python automatically closes the connection once operations inside the code block finish.

Read the file contents

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses= file.read()

# Display `ip_addresses`

print(ip_addresses)
```

To extract the data from the open file object, the `.read()` method is called on the `file` variable to convert the contents of `"allow_list.txt"` into a single string. This string is then saved to a new variable named `ip_addresses` so the file data can be accessed later in the script. Finally, the `print()` function is used to display the contents of `ip_addresses` to verify that the file was imported correctly.

Convert the string into a list

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Display `ip_addresses`

print(ip_addresses)
```

To convert the string of IP addresses into a format that can be modified, the `.split()` method is called on the `ip_addresses` variable. By default, `.split()` uses whitespace (including spaces and newlines) to separate the string into individual elements and convert it into a list. This resulting list is then reassigned back to `ip_addresses` so each IP address can be accessed and manipulated individually in subsequent steps.

Iterate through the remove list

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Display `element` in every iteration

    print(element)
```

To step through each item that needs to be removed, a `for` loop is header-configured to iterate through the `remove_list` list. The keyword `for` initiates the loop, while the `in` keyword specifies the sequence `remove_list` being traversed. On each iteration, the loop variable `element` holds the current IP address from `remove_list` so it can be evaluated against the allow list.

Remove IP addresses that are on the remove list

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)
```

Inside the loop, an `if` statement uses the `in` keyword to check whether the current `element` exists within the `ip_addresses` list. When this condition evaluates to `True`, the `.remove()` method is called on `ip_addresses` with `element` passed as its argument to delete that specific IP address from the allow list. Applying the `.remove()` method in this way is possible because there are no duplicate IP addresses in the `ip_addresses` list, ensuring each target entry is removed completely on its first match.

Update the file with the revised list of IP addresses

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

  # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

  ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

  # Build conditional statement
  # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

  # Rewrite the file, replacing its contents with `ip_addresses`

  file.write(ip_addresses)
```

To prepare the list of IP addresses for writing back to the file, the `.join()` method is applied to the string `"\n"` to concatenate all elements in `ip_addresses` into a single string separated by newlines. Next, a second `with` statement and the `open()` function are used with the mode parameter `"w"` to open the target file in write mode, safely overwriting its previous contents. Finally, the `.write()` method is called on the `file` object to write the updated IP address string into `"allow_list.txt"`.

Summary

This algorithm automates the management of access control lists by opening and updating the `"allow_list.txt"` file in Python. Using a `with` statement and the `open()` function in read mode alongside the `.read()` and `.split()` methods, the script converts raw file text into a list of IP addresses. It then iterates through a separate `remove_list` using a `for` loop,

checking whether each IP address exists in the allow list. When a match is found, the `.remove()` method deletes the target IP address from the list, which executes cleanly because there are no duplicate entries. Lastly, the updated list is converted back into a newline-separated string using `.join()` and written back to `"allow_list.txt"` using the `.write()` method in write mode.